

Project: Scalable Web Application with Load Balancing

Services Used

Amazon VPC, EC2, Bastion Host, Application Load Balancer, Auto Scaling Group, Amazon RDS MySQL, Amazon SNS, PM2, Security Groups, Internet Gateway, NAT Gateway, Route Tables, S3

Description

This project deploys a Node.js user authentication web application on AWS using a secure and scalable architecture. The backend application runs on EC2 instances in private subnets, the database runs on Amazon RDS MySQL, and incoming traffic is distributed using an Application Load Balancer. A Bastion host is used for secure SSH access to private instances. Auto Scaling automatically increases or decreases the number of backend servers based on CPU utilization, and SNS is used to send scaling notifications.

Definition

A scalable web application with load balancing is an application architecture in which traffic is distributed across multiple servers and the number of servers is adjusted automatically depending on load. This improves availability, security, and performance.

Scope of the Project

- Deploy a Node.js authentication application on AWS
- Create secure network architecture using VPC and subnets
- Use Bastion host for secure access
- Use private backend server for application hosting
- Use Amazon RDS for MySQL database
- Use Application Load Balancer for traffic distribution
- Use Auto Scaling Group for automatic scaling
- Use SNS topic for notifications
- Optionally host frontend using Amazon S3

This Project Covers

- Secure VPC design
- Public and private subnet setup
- Security group configuration
- Bastion to backend SSH model
- Backend deployment using SCP and SSH
- MySQL database connection using RDS
- Process management using PM2
- Health check configuration
- Load balancing with ALB
- Automatic scaling with ASG
- Scaling notifications with SNS

Methodology

1. Secure Multi-Tier AWS Architecture (Implemented Method)

- Uses VPC, Bastion host, private backend EC2, RDS, ALB, Auto Scaling, SNS
- Backend is not directly public

- Traffic comes through ALB only
- SSH is performed through Bastion only
- RDS is private and accessible only from backend security group
- Auto Scaling uses CPU-based target tracking

2. Single Public EC2 Deployment (Alternative Method)

- Application can be directly deployed on one public EC2
- Easier but less secure
- No load balancing
- No scaling
- Not used in this project because it is not production-style

AWS Services Used in the Project

1. Amazon VPC

Definition: Amazon VPC is a logically isolated virtual network in AWS.

Purpose: To create a secure network environment.

Role: Hosts public and private subnets, route tables, EC2 instances, ALB, and RDS.

2. Amazon EC2

Definition: Amazon EC2 provides virtual machines in the cloud.

Purpose: To run the backend Node.js application.

Role: Bastion EC2 is used for SSH access and backend EC2 runs the application.

3. Bastion Host

Definition: A Bastion host is a public EC2 instance used as a secure jump server.

Purpose: To access private instances securely.

Role: Used to copy files and SSH into the backend EC2 in private subnet.

4. Application Load Balancer

Definition: ALB distributes incoming HTTP traffic across multiple targets.

Purpose: To route traffic to backend instances.

Role: Public entry point for users.

5. Auto Scaling Group

Definition: Auto Scaling Group automatically launches or terminates EC2 instances based on load.

Purpose: To provide scalability and availability.

Role: Creates multiple backend instances when CPU increases.

6. Amazon RDS MySQL

Definition: Amazon RDS is a managed relational database service.

Purpose: To store user registration and login data.

Role: Stores users table and authentication data.

7. Amazon SNS

Definition: Amazon SNS is a notification service for email and message delivery.

Purpose: To notify scaling events.

Role: Sends alerts when instances launch or terminate.

8. Amazon S3

Definition: Amazon S3 is object storage used for static files.

Purpose: To host frontend files.

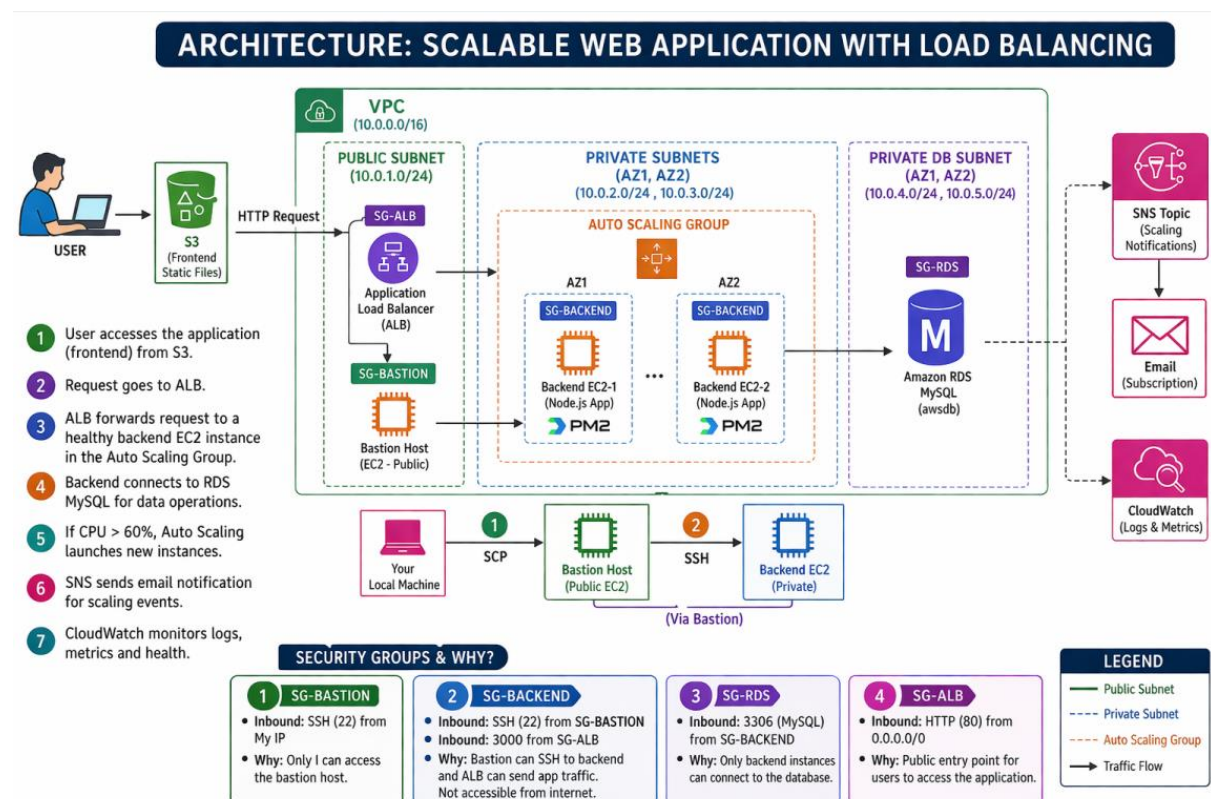
Role: Optional final step to host frontend separately.

9. PM2

Definition: PM2 is a process manager for Node.js applications.

Purpose: To keep the Node.js server running continuously.

Role: Automatically restarts the application if it stops.



Project Application Details

Application Name

UserAuthenticate-app

Backend Technology

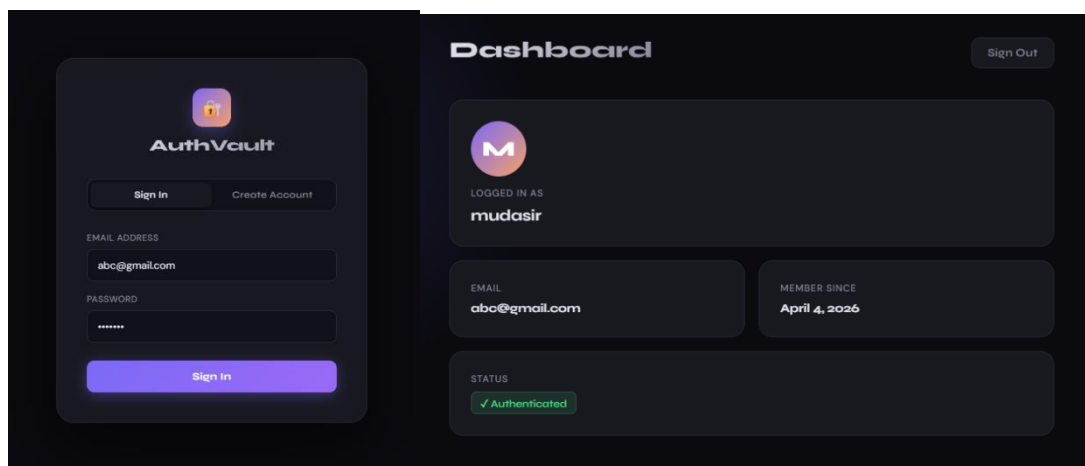
- Node.js
- Express.js
- mysql2/promise
- bcryptjs
- express-session

Database

- MySQL
- Database name: awsdbs
- Table: users

Main Features

- User registration
- User login
- Session management
- Logout
- Protected dashboard
- Health check endpoint

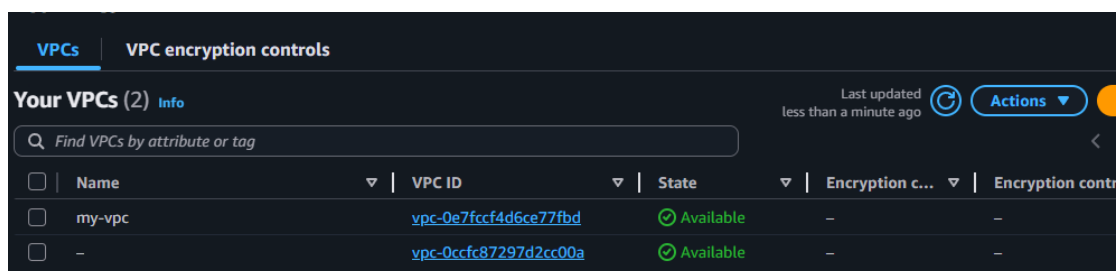


Implementation Steps:

Step 1: Create Custom VPC

Created a custom VPC for isolated network setup.

Purpose: A custom VPC is required to separate public-facing resources from private resources.



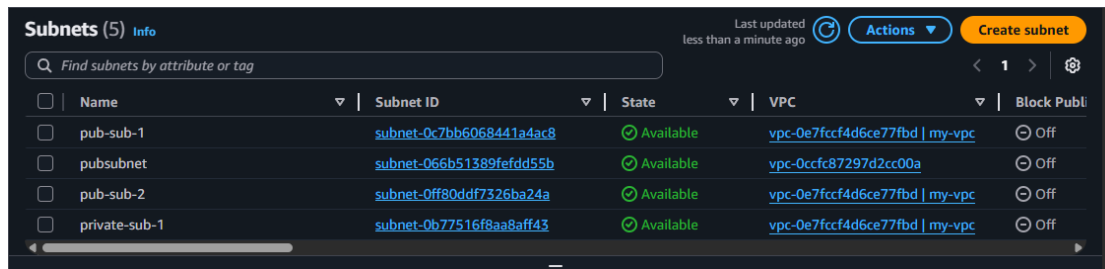
Step 2: Create Subnets

Created:

- Public subnet for Bastion host and ALB
- Private subnet for backend EC2
- Private subnets for Auto Scaling Group and RDS

Purpose:

- Public subnet is needed for internet-facing resources
- Private subnet is needed to secure backend and database servers



Subnets (5) Info

Last updated less than a minute ago

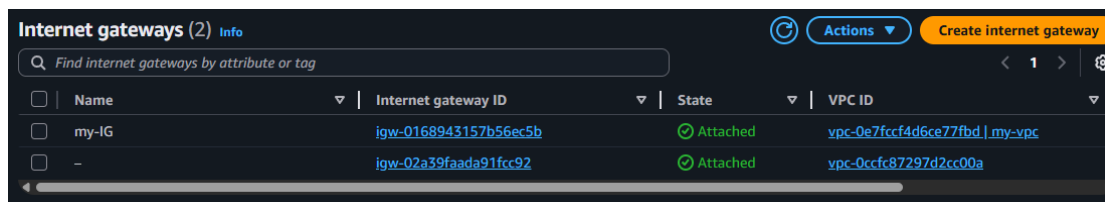
Find subnets by attribute or tag

☐	Name	Subnet ID	State	VPC	Block Publi
☐	pub-sub-1	subnet-0c7bb6068441a4ac8	Available	vpc-0e7fccf4d6ce77fbd my-vpc	Off
☐	pubsubnet	subnet-066b51389fefd55b	Available	vpc-0ccfc87297d2cc00a	Off
☐	pub-sub-2	subnet-0ff80ddf7326ba24a	Available	vpc-0e7fccf4d6ce77fbd my-vpc	Off
☐	private-sub-1	subnet-0b77516f8aa8aff43	Available	vpc-0e7fccf4d6ce77fbd my-vpc	Off

Step 3: Attach Internet Gateway

Attached an Internet Gateway to the VPC.

Purpose: This allows public subnet resources like Bastion and ALB to communicate with the internet.



Internet gateways (2) Info

Find internet gateways by attribute or tag

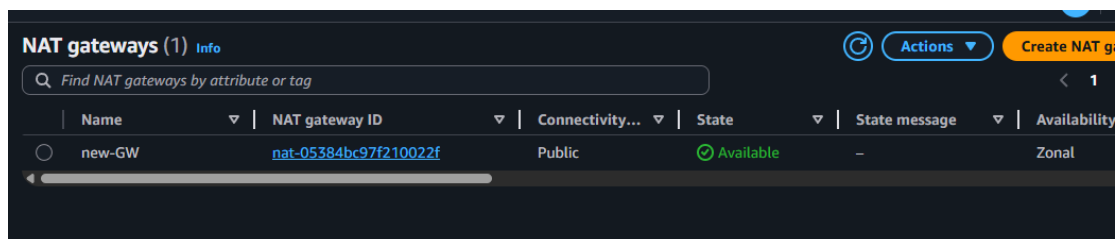
☐	Name	Internet gateway ID	State	VPC ID
☐	my-IG	igw-0168943157b56ec5b	Attached	vpc-0e7fccf4d6ce77fbd my-vpc
☐	-	igw-02a39faada91fcc92	Attached	vpc-0ccfc87297d2cc00a

Step 4: NAT gateway:

- Created NAT Gateway in public subnet
- Attached Elastic IP
- Updated private route table to send internet-bound traffic through NAT Gateway

Note: Private backend EC2 used NAT Gateway for outbound internet access to install packages like Node.js, npm dependencies, and MySQL client without assigning a public IP.

Purpose:: This allows private backend EC2 instances to download packages and updates without giving them public IP addresses.



NAT gateways (1) Info

Find NAT gateways by attribute or tag

☐	Name	NAT gateway ID	Connectivity...	State	State message	Availability
☐	new-GW	nat-05384bc97f210022f	Public	Available	-	Zonal

Step 4: Configure Route Tables

Configured public route table:

- 0.0.0.0/0 → Internet Gateway

Configured private route table:

- 0.0.0.0/0 → NAT Gateway

Purpose: Public route table is needed for internet access.
Private route table keeps backend isolated.

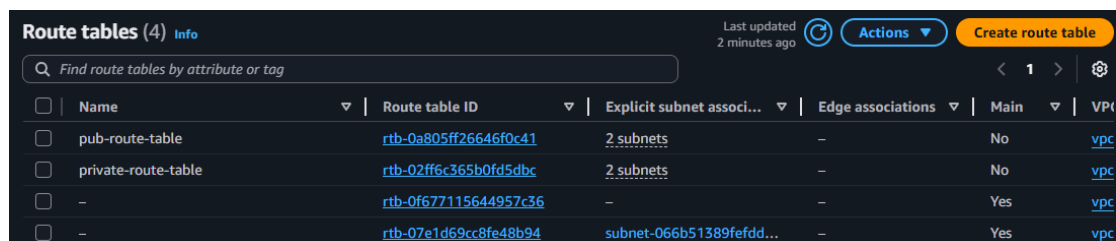
Step 4: Route table association

· **Public route table** associated with:

- public subnet for Bastion
- public subnet(s) for ALB
- subnet containing NAT Gateway

· **Private route table** associated with:

- backend private subnet
- Auto Scaling private subnet(s)
- RDS private subnet(s), if you used separate DB subnet group



The screenshot shows the AWS Route Tables console. At the top, it says 'Route tables (4) Info' and 'Last updated 2 minutes ago'. There are buttons for 'Actions' and 'Create route table'. A search bar says 'Find route tables by attribute or tag'. Below is a table with columns: Name, Route table ID, Explicit subnet associations, Edge associations, Main, and VPC. The table lists four route tables: 'pub-route-table' (ID: rtb-0a805ff26646f0c41, 2 subnets, No, vpc-0a805ff26646f0c41), 'private-route-table' (ID: rtb-02ff6c365b0fd5dbc, 2 subnets, No, vpc-02ff6c365b0fd5dbc), and two other tables with hyphen names (IDs: rtb-0f677115644957c36 and rtb-07e1d69cc8fe48b94, both with 1 subnet, Yes, vpc-0f677115644957c36 and vpc-07e1d69cc8fe48b94 respectively).

☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
☐	pub-route-table	rtb-0a805ff26646f0c41	2 subnets	–	No	vpc-0a805ff26646f0c41
☐	private-route-table	rtb-02ff6c365b0fd5dbc	2 subnets	–	No	vpc-02ff6c365b0fd5dbc
☐	–	rtb-0f677115644957c36	–	–	Yes	vpc-0f677115644957c36
☐	–	rtb-07e1d69cc8fe48b94	subnet-066b51389fefdd...	–	Yes	vpc-07e1d69cc8fe48b94

Step 5: Create Security Groups:

1. Bastion Security Group

Inbound rules:

- SSH (22) → My IP

Purpose: Only my laptop should access Bastion.

2. Backend Security Group

Inbound rules:

- SSH (22) → Bastion Security Group
- Custom TCP (3000) → ALB Security Group

Purpose:

- Bastion can SSH into backend
- ALB can send application traffic to backend
- Backend is not directly open to internet

3. RDS Security Group

Inbound rules:

- MySQL/Aurora (3306) → Backend Security Group

Purpose: Only backend application should connect to MySQL.

4. ALB Security Group

Inbound rules:

- HTTP (80) → 0.0.0.0/0

Purpose: ALB is public and accepts user traffic.

Security group setup is a major part of the project and should be described clearly, just like your friend explained permissions and role configuration in detail.

☐	-	sg-0d1c71197cd520de9	default	vpc-0ccfc87297d2cc00a 
☐	rds-sg	sg-0fb11abd61ccff4d3	rds-sg	vpc-0e7fccf4d6ce77fbd 
☐	backend-sg	sg-0dc097b11c789a695	Backend SG	vpc-0e7fccf4d6ce77fbd 
☐	alb sg	sg-0de999462c3b7eec2	ALB SG	vpc-0e7fccf4d6ce77fbd 
☐	bastion sg	sg-0bff61119b2215e3b	Bastion SG	vpc-0e7fccf4d6ce77fbd 

Step 6: Launch Bastion EC2

Created a Bastion EC2 instance in the public subnet.

Configuration:

- Public subnet
- Public IP enabled
- Bastion Security Group attached
- Key pair: awsprojectkey.pem

Purpose: Bastion is the only public server used for SSH access to the backend private server.

Step 7: Launch Backend EC2

Created backend EC2 instance in private subnet.

Configuration:

- Private subnet
- No public IP
- Backend Security Group attached
- Same key pair used

Purpose: Backend should remain private and should not be directly accessible from internet.

Step 8: Connect from Local Machine to Bastion

Used SSH from local machine using (git-bash-terminal):

```
ssh -i awsprojectkey.pem ec2-user@BASTION-PUBLIC-IP
```

Step 9: Copy Project from Local Machine to Bastion

Used SCP from local machine:

```
scp -i awsprojectkey.pem -r aws-project ec2-user@BASTION-PUBLIC-IP:~
```

Also copied key file to Bastion:

```
scp -i awsprojectkey.pem awsprojectkey.pem ec2-user@BASTION-PUBLIC-IP:~
```

Purpose: The project folder must first reach Bastion because backend is private. The key file is needed on Bastion to connect to backend EC2.

Step 10: Set Key Permission on Bastion

Ran:

```
chmod 400 awsprojectkey.pem
```

Purpose: Private key file must have secure permissions to be usable for SSH.

Step 11: Connect from Bastion to Backend

Used SSH from Bastion:

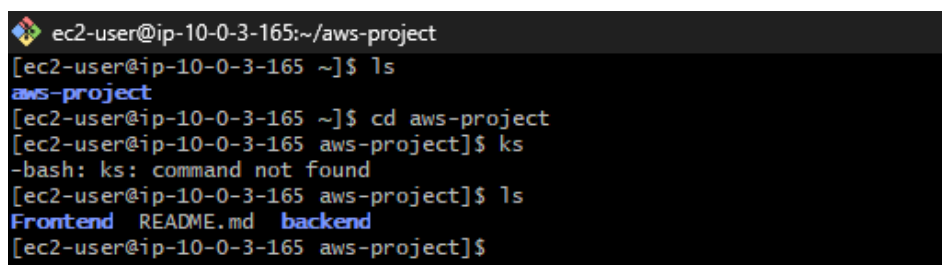
```
ssh -i awsprojectkey.pem ec2-user@<BACKEND-PRIVATE-IP>
```

Purpose: Backend is in private subnet and cannot be accessed directly from local system.

Step 12: Copy Project from Bastion to Backend

Used SCP from Bastion:

```
scp -i awsprojectkey.pem -r aws-project ec2-user@<BACKEND-PRIVATE-IP>:~
```



```
ec2-user@ip-10-0-3-165:~/aws-project
[ec2-user@ip-10-0-3-165 ~]$ ls
aws-project
[ec2-user@ip-10-0-3-165 ~]$ cd aws-project
[ec2-user@ip-10-0-3-165 aws-project]$ ks
-bash: ks: command not found
[ec2-user@ip-10-0-3-165 aws-project]$ ls
Frontend  README.md  backend
[ec2-user@ip-10-0-3-165 aws-project]$
```

Purpose: The backend application files must be present on backend EC2 for deployment.

- Your current server.js does **not** contain /health...Add this above your app.post('/api/register...):

```
app.get('/health', (req, res) => {
  res.status(200).send('OK');
});
```

Purpose: I created a /health endpoint so that the Application Load Balancer can perform health checks and route traffic only to healthy instances.

```
// --- ROUTES ---
app.get('/health', (req, res) => {
  res.status(200).send('OK');
});

// Register
app.post('/api/register', async (req, res) => {
  const { username, email, password } = req.body;
  if (!username || !email || !password)
```

Step 13: Install Node.js on Backend EC2

Ran:

```
sudo yum update -y
sudo yum install -y nodejs
node -v
npm -v
```

Step 14: Run Backend Application:

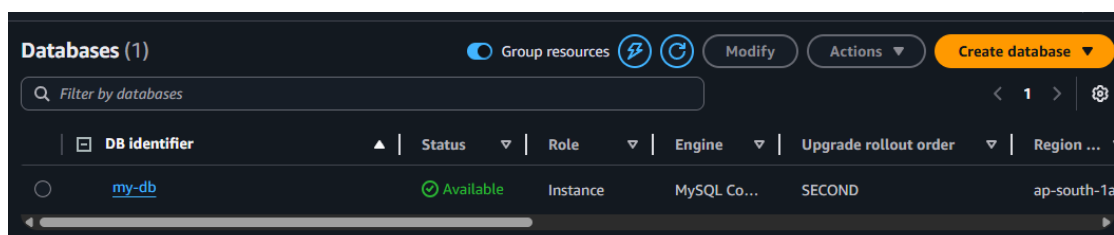
```
cd aws-project/backend
npm install
node server.js
```

This installs dependencies and starts the application.

Step 15: Create Amazon RDS MySQL

Created an RDS DB instance with:

- Engine: MySQL
- VPC: same project VPC
- Private subnet
- Public access: No
- Security group: RDS SG
- Username: admin
- Password: mudasirbhat
- Created/used an **RDS DB subnet group** with private subnets in multiple Availability Zones.



Purpose:: RDS provides managed MySQL for secure and scalable database hosting.

Step 16: Create Database Inside RDS

Connected from backend EC2 using MySQL client.

Installed client:

```
sudo yum install -y mariadb105
```

Connected to RDS:

```
mysql -h <RDS-ENDPOINT> -u admin -p
```

Created database:

```
CREATE DATABASE awsdB;  
SHOW DATABASES;  
EXIT;
```

Purpose: Creating the RDS instance does not automatically create the application database unless specified.

```
Database changed  
MySQL [awsdb]> SHOW TABLES;  
+-----+  
| Tables_in_awsdb |  
+-----+  
| users            |  
+-----+  
1 row in set (0.001 sec)  
  
MySQL [awsdb]> SELECT * FROM users;  
+-----+-----+-----+-----+-----+  
| id | username | email | password | created_at |  
+-----+-----+-----+-----+-----+  
| 1 | as;1kd1;aksd | asd;1askdkaspkd | $2a$10$2VLzdTxV3wmGUoQm/QV6N.DcpPBhFTIm9jJ6Vt6Xtn5XQM.Ko5HES | 2026-04-04 08:08:02 |  
| 2 | mudasir | abc@gmail.com | $2a$10$5T5b1CRFE48h76XAGXKeRuNKs7C6htIwFKDbNpRrQ80o69RkxjKmS | 2026-04-04 10:29:35 |  
+-----+-----+-----+-----+-----+  
2 rows in set (0.000 sec)  
  
MySQL [awsdb]>
```

Step 17: Update Backend Code for RDS

Updated `server.js` database connection:

```
const pool = mysql.createPool({  
  host: '<RDS-ENDPOINT>',  
  user: 'admin',  
  password: 'YOUR_PASSWORD',  
  database: 'awsdb',  
  port: 3306  
});
```

Purpose:: The backend must connect to RDS, not localhost.

Step 18: Health Check Testing:

Tested application on backend EC2:

```
curl http://localhost:3000/health
```

Expected output: OK

Purpose: To check that the application is running properly and to allow ALB to perform health checks.
Step

- **Configure npm start:** Runs your app using package.json,, · Stops when you close terminal -Not good for server
- **Configure PM2 :** Runs app in background (like a service), Keeps app running -Auto restart if crash -Works after reboot

Installed PM2:

`sudo npm install -g pm2`

Started application:

`pm2 start server.js --name auth-app`

Saved process:

`pm2 save`

Enabled auto start after reboot:

`pm2 startup`

`pm2 save`

`Pm2 status`

Give status

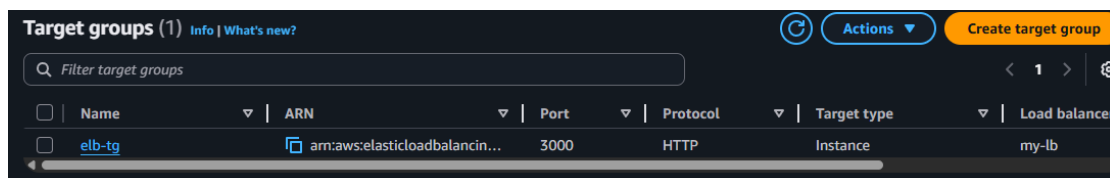
Purpose: PM2 keeps the application running in the background and automatically restarts it if the server reboots or the application crashes.

Step 19: Create Target Group

Created target group with:

- Target type: Instances
- Protocol: HTTP
- Port: 3000
- Health check path: /health

Registered backend EC2 instance.

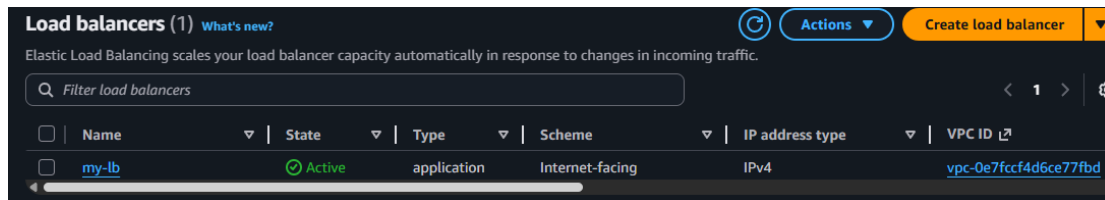


Purpose: The ALB forwards traffic to a target group, and health checks ensure traffic only goes to healthy instances.

Step 20: Create Application Load Balancer

Created ALB with:

- Internet-facing scheme
- Two public subnets
- ALB Security Group
- Listener: HTTP 80
- Forward to created target group



Purpose::ALB becomes the public entry point for the application and distributes traffic to backend instances.

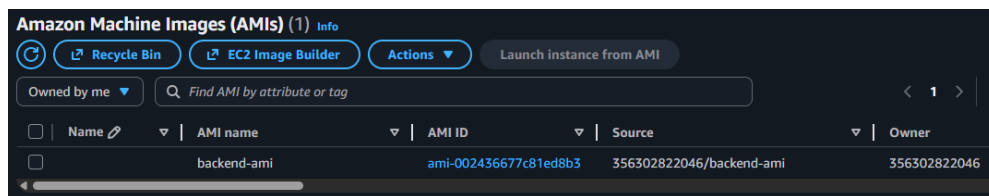
Step 21: Test ALB

Opened in browser: <http://<ALB-DNS-NAME>/health>

Purpose: To verify that ALB is successfully routing traffic to backend application.

Step 22: Create AMI from Backend EC2

Created an Amazon Machine Image from the configured backend EC2.

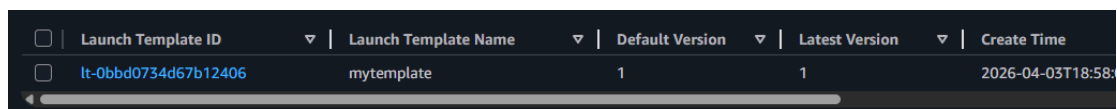


Purpose:: The AMI is used as a template for launching identical backend servers in Auto Scaling.

Step 23: Create Launch Template

Created Launch Template using:

- Backend AMI
- Backend Security Group
- Same key pair
- Private subnet deployment model



Purpose::Launch Template defines how new backend instances are created automatically.

Step 24: Create Auto Scaling Group

Created ASG with:

- Launch Template
- Private subnets
- Attached target group
- Min: 1
- Desired: 2
- Max: 3

Desired capacity type
Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for attributes.

Units (number of instances) ▼

Desired capacity
Specify your group size.

2

Scaling limits
Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity **Max desired capacity**

1 3

Equal or less than desired capacity Equal or greater than desired capacity

Purpose:: ASG ensures backend capacity can increase or decrease automatically.

Step 25: Configure Scaling Policy

Used target tracking scaling policy:

- Metric: Average CPU utilization
- Target value: 60%
- Instance warmup: 300 seconds

Purpose:: When CPU usage rises above target, ASG launches more instances. When load drops, it removes extra instances.

Step 26: Create SNS Topic

Created SNS topic and email subscription for notifications.

Purpose:: SNS provides email notifications for scaling events.

Step 27: Attach SNS Notifications to Auto Scaling Group

Configured Auto Scaling notifications for:

- Instance launch
- Instance launch error
- Instance terminate
- Instance terminate error

Topics (1)			
Search Edit Delete Publish message Create			
Name	Type	ARN	
my-new-top	Standard	arn:aws:sns:ap-south-1:356302822046:	

Purpose:: This sends alerts whenever scaling actions happen.

Navigation: Details | Integrations | Automatic scaling | **Instance management** | Instance refresh | Activity | Monitor

Instances (2)

Filter instances

☐	Instance ID	Lifecycle	Instanc...	Weight...	Launch...	Availab...	Health ...
☐	i-0346418bc62a3dfc0	InService	t3.micro	-	mytemplate	aps1-az3 (...)	Healthy
☐	i-092d683cdd856952a	InService	t3.micro	-	mytemplate	aps1-az1 (...)	Healthy

Instance state = running | Clear filters

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability
☐		i-0346418bc62a3dfc0	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1k
☐	Backend EC2	i-0c4908898355b7b64	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1a
☐	Bastion EC2	i-07a22fb338d43e60e	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1a
☐		i-092d683cdd856952a	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1a

my-lb

Rules (1): Priority default, Forward to target group

Target groups (1): Instance, HTTP, elb-tg, 3 targets

Targets (3):

- [i-0346418bc62a3dfc0](#) Port 3000: Healthy
- [i-092d683cdd856952a](#) Port 3000: Healthy
- [i-0c4908898355b7b64](#) Port 3000: Healthy

Step 28: S3 Frontend Hosting

steps:

- Create S3 bucket
- Upload frontend static files
- Enable static website hosting
- Update API URL to ALB DNS

Purpose::Frontend can be served from S3 while backend APIs are served from ALB.

Navigation: Objects | Metadata | Properties | Permissions | Metrics | Management | Access Points

Objects (1)

Copy S3 URI | Copy URL | Download | Open | Delete | Actions | Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	index.html	html	April 4, 2026, 13:35:11 (UTC+05:30)	17.1 KB	Standard

Troubleshooting:

Issue 1: SCP Failed

Solution:

- Ensure key file exists on Bastion
- Run `chmod 400 awsprojectkey.pem`

Issue 2: ECONNREFUSED 127.0.0.1:3306

Solution:

- RDS was not configured yet
- Replace localhost with RDS endpoint

Issue 3: Unknown database

Solution:

- Connect to RDS and create `awsdb`

Issue 4: Cannot GET /health

Solution:

- Confirm `server.js` has `/health` route
- Restart correct Node process
- Remove older conflicting process

Issue 5: Target Group Unhealthy

Solution:

- Confirm `/health` path works
- Confirm backend SG allows port 3000 from ALB SG
- Confirm app is running on port 3000

Conclusion

This project successfully deploys a secure and scalable web application on AWS using a multi-tier architecture. The application backend runs on private EC2 instances, the database is hosted on Amazon RDS, and traffic is distributed through an Application Load Balancer. Auto Scaling improves availability and performance by launching additional instances during high load. Bastion host ensures secure administration, and SNS notifications provide visibility into scaling events. This architecture follows real-world cloud deployment practices and demonstrates security, scalability, and high availability.

